Project 4

The solution to this project should be in one single Jupyter notebook.

- The notebook should run as a program.

All cells should execute correctly in the order in which they appear.

- For each assignment, use a separate header in the notebook.

- Use text cells to answer the questions of the assignments.

- Write complete, grammatically correct sentences to answer the questions.

Explanations matter just as much as the numerical results and the code.

Submit your project before the deadline to gradescope.

The name of your notebook should be FirstNameLastNameProjectNumber.ipynb, where you replace FirstNameLast with your own first and last name.

The first cell in your solution notebook should be a text cell that lists your name.

If you have questions about the project, you are welcome in the office hours.

Assignment 1

Consider the initial value problem

$$y' = \frac{t+y}{t-y} \text{ on } [0,3] \text{ with } y(0) = 1.$$

The exact solution is $y(t) = 3e^{-t/2} - t - 1$ Use Euler's method with $h = 1, 1/2, 1/4$ and $1/8$ to approximate $y(3)$. Graph and label the approximations and the exact solution on the same graph.

In [1]:
```
using Plug, Plug.gnuplot("Euler")
using Plots

# Define the differential equation dy/dt = (t + y)/ (t
f(t, y) = (t + y) / (t - y)

# Exact solution
exact_solution(t) = 3 * exp(-t / 2) - 2 * t - 1

# Euler's method iteration
function iterate(half, t, y, dt, f)
    if half == 0
        t = [mapsto(t, t), mapsto(t + dt, t)]
        y = [mapsto(y, y), mapsto(y + dt * f(t, y), y)]
    else
        t = [mapsto(t, t), mapsto(t + dt/2, t)]
        y = [mapsto(y, y), mapsto(y + dt/2 * f(t, y), y)]
    end
    return t, y
end

# Changed and initial condition
h = 1.0
t, y = 0
dt = h
half = 0

# Step sizes
step_sizes = [1, 0.5, 0.25, 0.125]

# Plot the exact solution
plot(exact_solution)
# Loop through different step sizes and plot each
combined_plot = plot!(exact_solution, label="Exact solution", color=black, lw=2, xlabel="t", ylabel="y(t)", title="Euler's Method vs Exact Solution")

# Add Euler's method approximations for each step size
for dt in step_sizes
    t, y = iterate(0, t, y, dt, f)
    for i in 1:length(t) - 1
        plot!(combined_plot, t, y, label="Euler (h=$dt)")
    end
end

# Display the combined plot
display(combined_plot)
```

```
Rendering package versions...
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
```

The code implements Euler's Method to approximate the solution of the differential equation shown above, given the exact solution y(t). The function autiterate that iteratly computes the approximate values for y over interval [a,b] using step size h which in this case is 1, 1/2, 1/4, and 1/8. The plot shows that the larger step size produce less accurate results with greater deviations from the exact solution as t increases. Smaller step sizes provide a closer match to the exact solution, demonstrating how reducing h improves accuracy.

Assignment 2

The trapezoidal method is an implicit method

$$w_{i+1} = w_i + \frac{h}{2}\left(f(t_i, w_i) + f(t_{i+1}, w_{i+1})\right)$$

It can be combined with an explicit method such as Euler's method

$$w_{i+1} = w_i + hf(t_i, w_i).$$

to get a new better method

$$w_{i+1} = w_i + \frac{h}{2}\left(f(t_i, w_i) + f(t_{i+1}, w_i + hf(t_i, w_i))\right).$$

One says that Euler's method is used to predict the value of w_{i+1} which is then used in the implicit method. This is called a predictor corrector method. Combine the Adams Bashforth Four Step Method (See Sauer formula 6.66) and the Adams-Moulton Three Step Method (See Sauer formula 6.75) in a predictor corrector method. Use step size 0.05. Plot the approximation and the correct solution of

$$y' = 4y + t^2 \text{ with } y(0) = 1,$$

on the interval [0, 5].

In [2]:
```
using Plug, Plug.gnuplot("Euler")
using Plots

# Define the differential equation y' = 4*y + t^2
f(t, y) = 4 * y + t^2

# Define the exact solution (use extreme method or analytical)
exact_solution(t) = exp(4 * t) - (t^2 + t)

# Euler's method for initial condition
function iterate_euler(half, t, y, dt, f)
    if half == 0
        return y + dt * f(t, y)
    end
end

# Adams Bashforth Four-Step Method (predictor)
function adams_bashforth(y_0, y_1, y_2, y_3, dt, f)
    return y + h / 24 * (-9 * f_values[4] + 37 * f_values[3] - 59 * f_values[2] + 55 * f_values[1])
end

# Adams Moulton Three-Step Method (corrector)
function adams_moulton(y_0, y_1, y_2, dt, f)
    return y + h / 24 * (-f_values[3] + 5 * f_values[2] + 19 * f_values[1] + 9 * f_values[0])
end

# Predictor-Corrector Method
function predictor_corrector(f, t0, y0, h, n_steps)
    # Initialize arrays to store results
    t = zeros(n_steps+1)  # Initialize time array
    y_pred = zeros(n_steps+1)  # Initialize storage for function values

    # Initial values
    t[1] = t0
    y_pred[1] = y0
    for i in 1:3
        y_pred[i+1] = rk4step(f, t[i], y[i], h)
        t[i+1] = t[i] + h
    end
    y_pred[4] = rk4step(f, t[4], y[4], h)

    # Predictor-corrector steps
    for i in 4:n_steps-1
        f_values = [f(t[i-3], y_pred[i-3]), f(t[i-2], y_pred[i-2]), f(t[i-1], y_pred[i-1]), f(t[i], y_pred[i])]
        y_predict = ab4step(y_pred[i-3], y_pred[i-2], y_pred[i-1], y_pred[i], h, f_values)
        t[i+1] = t[i] + h

        # Update function values
        f_new = f(t[i+1], y_predict)
        f_values = [f_values[2], f_values[3], f_values[4], f_new]

        # Corrector step
        y_corrected = am3step(y_pred[i-2], y_pred[i-1], y_pred[i], y_predict, h, f_values[2:4])

        # Store corrected value
        y_pred[i+1] = y_corrected
    end

    return t, y_pred
end

# Parameters
t0 = 0.0
y0 = 1.0
h = 0.05
n_steps = Int(1 / h)

# Solve the differential equation
t_values, y_values = predictor_corrector(f, t0, y0, h, n_steps)

# Exact solution
t_exact = 0:0.01:1
y_exact = exact_solution.(t_exact)

# Plot results
plot!(t_exact, y_exact, label="Exact Solution", color=black, lw=2, xlabel="t", ylabel="y(t)", title="Predictor-Corrector Method vs Exact Solution")
display(plot!(t_values, y_values, label="Predictor-Corrector (h=0.05)", marker=circle, lw=1.5))
```

```
Rendering package versions...
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
```

The provided code implements a Predictor-Corrector Method to solve the IVP over the interval [0, 5] with a step size of h = 0.05. This method combines Adams-Bashforth Four-Step Method (predictor) and the Adams-Moulton Three-Step Method (corrector). The first three steps are computed using the Trapezoidal Method for initialization to generate enough starting values for the multistep methods. The plot compares the numerical solution (red) from the Predictor-Corrector method to the exact solution (black). The numerical solution closely matches the exact solution for most of the interval but as t approaches 5, the numerical solution begins to deviate.

Assignment 3

For each linear BVP, use (or modify) the finite element code provided to solve the problem and plot the solution for $n = 10, 20, 40, \ldots, 640$. Compute the norm of the error. Make a log-log convergence plot of error versus n and compare graphically to second order convergence

$$(a) \quad -u'' + u = -8 + 15x^2 - x^4 \text{ u(0) = u(2) = 0 Exact solution } x^4(2-x)(x-1) \qquad (b) \quad -u'' + u(x^2 + u) = -x(3x\sin(x) - 5x\cos(x)) \text{ u(0) = u(2) = 0 Exact solution } -x^2\sin(x)$$

In [3]:
```
using Plug, Plug.gnuplot("LinearAlgebra")
using LinearAlgebra
using Plots

# Define the functions
k(x) = 1.0
q(x) = 1.0
f(x) = -8 + 15x^2 - x^4
u_exact(x) = x^4 * (2 - x) * (x - 1)

# Define the domain
a, b = 0.0, 2.0

# FEM function (from the provided code)
function fem(n, k, q, f, a, b, u)
    h = (b - a) / n
    x = a:h:b

    ne = (n - 1) - 1 + 1
    ne = (n - 1) - 1 + 1
    ne = 1:(n - 1) - 1 + 1

    ind(e) = e
    inod(e) = e
    xnod(e) = x[inod(e)] + x[inod(e) + 1]
    ke(e) = k(xnod(e))
    fe(e) = f(xnod(e))

    K = zeros(n, n, n)
    F = zeros(n, n)
    U = zeros(n)

    K[1, 1] = ke(1) / h
    F[1, 1] = fe(1) * h / 2
    K[1, 2] = -ke(1) / h

    K[n, n, 1] = ke(n) / h
    K[n, n, 1] = ke(n) / h + 1
    F[n, n] = fe(n) * h / 2

    for e in 2:ne
        K[e, e, 1] = ke(e) / h
        K[e, e, 1] = ke(e) / h + ke(e - 1) * h
        K[e, e-1, 1] = -ke(e) / h
        F[e, e] = fe(e) * h
    end

    # Solve the system
    for e in 1:ne
        U = U + K[:, :, e] \ F[:, e]
    end

    # Add boundary conditions
    U[1] = u(a)
    U[n] = u(b)

    return U, x, h
end

# Initialize variables
n_values = [10, 20, 40, 80, 160, 320, 640]
errors = Float64[]

# Loop through different mesh sizes
for n in n_values
    U, x, h = fem(n, k, q, f, a, b, u)
    exact = u_exact.(x)
    error = norm(U - exact, 2)
    push!(errors, error)
end

# Plot
plot(n_values, errors, xlabel="Number of elements", ylabel="Error", title="Solution for n=40")
```

```
Rendering package versions...
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
```

```
Norm of the error for code: 0.012324007177444
```

The code implements the finite element method (FEM) to solve the boundary value problem. It defined k(x), q(x), and f(x), assembles the global stiffness matrix, mass matrix, and load vector, and solves the system to compute the numerical solution. The numerical solution decreases significantly for the exact solution corresponding to the small error. The log-log convergence plot illustrates the relationship between error and n, confirming method's second order convergence through the linear slope.

In [4]:
```
using Plug, Plug.gnuplot("LinearAlgebra")
using LinearAlgebra
using Plots

# Define the functions for 3b
k(x) = 1.0
q(x) = 1
f(x) = 1
u_exact(x) = x^2 * (2 - x)

# Define the domain
a, b = 0.0, 2.0

# FEM function (from the provided code)
function fem(n, k, q, f, a, b, u)
    h = (b - a) / n
    x = a:h:b

    ne = (n - 1) - 1 + 1
    ne = (n - 1) - 1 + 1
    ne = 1:(n - 1) - 1 + 1

    ind(e) = e
    inod(e) = e
    xnod(e) = x[inod(e)] + x[inod(e) + 1]
    ke(e) = k(xnod(e))
    fe(e) = f(xnod(e))

    K = zeros(n, n, n)
    F = zeros(n, n)
    U = zeros(n)

    K[1, 1] = ke(1) / h
    F[1, 1] = fe(1) * h / 2
    K[1, 2] = -ke(1) / h

    K[n, n, 1] = ke(n) / h
    K[n, n, 1] = ke(n) / h + 1
    F[n, n] = fe(n) * h / 2

    for e in 2:ne
        K[e, e, 1] = ke(e) / h
        K[e, e, 1] = ke(e) / h + ke(e - 1) * h
        K[e, e-1, 1] = -ke(e) / h
        F[e, e] = fe(e) * h
    end

    # Solve the system
    for e in 1:ne
        U = U + K[:, :, e] \ F[:, e]
    end

    # Add boundary conditions
    U[1] = u(a)
    U[n] = u(b)

    return U, x, h
end

# Initialize variables
n_values = [10, 20, 40, 80, 160, 320, 640]
errors = Float64[]

# Loop through different mesh sizes
for n in n_values
    U, x, h = fem(n, k, q, f, a, b, u)
    exact = u_exact.(x)
    error = norm(U - exact, 2)
    push!(errors, error)
end

# Plot
plot(n_values, errors, xlabel="Number of elements", ylabel="Error", title="Solution for n=40")

# Compute the norm of the error
error_norm = errors[length(n_values)]
println("Norm of the error for code: $error_norm")

# Log-log plot of error vs n
plot(n_values, errors, xscale=:log10, yscale=:log10, marker=:o,
    xlabel="n", ylabel="Error", title="Log-Log Convergence Plot")
```

```
Rendering package versions...
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
No Changes to `~/Library/Mobile Documents/com~apple~CloudDocs/Jupyt
```

```
Norm of the error for code: 1.8E-15
```

The code implements the finite element method (FEM) to solve the boundary value problem. It defined k(x), q(x), and f(x), assembles the global stiffness matrix, mass matrix, and load vector, and solves the system to compute the numerical solution. The numerical solution decreases significantly for the exact solution, especially as n approaches 640 suggesting that the method struggles with the nonlinearity. The error norm is also significantly large which shows the challenges of solving problems with nonlinear terms using FEM with linear basis functions. The log-log convergence plot shows a decreasing error as n increases, but the slope does not indicate second order convergence which could be from the complexity of the exact solution. Further steps for this problem could be to increase n and use higher order basis functions such as quadratic or cubic functions.

Assignment 4

For each linear BVP, use the finite element code provided to solve the problem and plot the solution for $n = 80$. Then for each $n = 10, 20, 40, \ldots, 640$, compute the norm of the error. Make a log-log convergence plot of error versus n and compare graphically to second order convergence.

$$(a) \quad -u'' + u = -8 + 15x^2 - x^4 \text{ u(0) = u(2) = 0 Exact solution } x^4(2-x)(x-1) \qquad (b) \quad -u'' + u(x^2 + u) = -x(3x\sin(x) - 5x\cos(x)) \text{ u(0) = u(2) = 0 Exact solution } -x^2\sin(x)$$

In [5]:
```
using LinearAlgebra
using Plots

# Define the functions for 4a
k(x) = 1.0
q(x) = 1
f(x) = -8 + 15x^2 - x^4
u_exact(x) = x^4 * (2 - x) * (x - 1)

# Define the domain
a, b = 0.0, 2.0

# FEM function (from the provided code)
function fem(n, k, q, f, a, b, u)
    h = (b - a) / n
    x = a:h:b

    ne = (n - 1) - 1 + 1
    ne = (n - 1) - 1 + 1
    ne = 1:(n - 1) - 1 + 1

    ind(e) = e
    inod(e) = e
    xnod(e) = x[inod(e)] + x[inod(e) + 1]
    ke(e) = k(xnod(e))
    fe(e) = f(xnod(e))

    K = zeros(n, n, n)
    F = zeros(n, n)
    U = zeros(n)

    K[1, 1] = ke(1) / h
    F[1, 1] = fe(1) * h / 2
    K[1, 2] = -ke(1) / h

    K[n, n, 1] = ke(n) / h
    K[n, n, 1] = ke(n) / h + 1
    F[n, n] = fe(n) * h / 2

    for e in 2:ne
        K[e, e, 1] = ke(e) / h
        K[e, e, 1] = ke(e) / h + ke(e - 1) * h
        K[e, e-1, 1] = -ke(e) / h
        F[e, e] = fe(e) * h
    end

    # Solve the system
    for e in 1:ne
        U = U + K[:, :, e] \ F[:, e]
    end

    # Add boundary conditions
    U[1] = u(a)
    U[n] = u(b)

    return U, x, h
end

# Initialize variables
n_values = [10, 20, 40, 80, 160, 320, 640]
errors = Float64[]

# Loop through different mesh sizes
for n in n_values
    U, x, h = fem(n, k, q, f, a, b, u)
    exact = u_exact.(x)
    error = norm(U - exact, 2)
    push!(errors, error)
end

# Plot
plot(n_values, errors, xlabel="Number of elements", ylabel="Error", title="Solution for n=40")

# Compute the norm of the error
error_norm = errors[length(n_values)]
println("Norm of the error for code: $error_norm")

# Log-log plot of error versus n
plot(n_values, errors, xscale=:log10, yscale=:log10, marker=:o,
    xlabel="n", ylabel="Error", title="Log-Log Convergence Plot for 4b")
```



```
Norm of the error for code: 1.8E-15
```

The code implements the finite element method (FEM) to solve the boundary value problem. It defined k(x), q(x), and f(x), assembles the global stiffness matrix, mass matrix, and load vector, and solves the system to compute the numerical solution. Similar to 3b, the numerical solution also decreases from the exact solution, suggesting challenges to capture the problem's complexity with linear basis function, having an error norm of 1E-02. The log-log convergence plot shows a consistent decrease in error as n increases but the convergence is slower than the expected second order rate which could be due to nonlinearity.